

Final Project Deliverable

Problem Statement:

Children in Lansing's low-income, pre-1978 rental housing remain at higher risk of elevated blood lead levels, despite statewide declines in lead exposure. This project aims to examine whether Lead Safe Lansing's (LSL) and local governments' abatement efforts are reaching the socioeconomically disadvantaged neighborhoods where young children face the greatest risk between 2010 and 2022.

Research Questions:

1. Research Question 1: Among ZIP codes in the City of Lansing, Michigan, from 2010 to 2022, how does Lead Safe Lansing's greater concentration of projects in higher-poverty, higher-EBLL ZIP codes associate with reductions in the percentage of children under age 6 with elevated blood lead levels compared with ZIP codes receiving lower concentrations of projects?
2. Research Question 2: For Lead Safe Lansing residential abatement projects completed in Lansing, Michigan, between 2010 and 2022, how are rehabilitation costs related to changes in general price inflation over time?

```
In [1]: # Importing libraries
import pandas as pd # for data analysis, and working with DF
import numpy as np # to work with number
import matplotlib.pyplot as plt # visualizations
import seaborn as sns # visualiaztions
import censusgeocode as cg # Used for address search for RQ1
import os #help identify file
from scipy import stats # statistical tests
import statsmodels.api as sm
import matplotlib.lines as mlines
import statsmodels.formula.api as smf
from statsmodels.stats.multicomp import pairwise_tukeyhsd #Tukey HSD test for p
```

RQ1: Among ZIP codes in the City of Lansing, Michigan, from 2010 to 2022, how does Lead Safe Lansing's greater concentration of projects in higher-poverty, higher-EBLL ZIP codes associate with reductions in the percentage of children under age 6 with elevated blood lead levels compared with ZIP codes receiving lower concentrations of projects?

Datasets to be used:

1. LSL Projects Dataset
2. NaNDA Socioeconomic Data
3. EBLI datasets

```
In [2]: # 1) Blood-lead data
bl = pd.read_csv("data_151559.csv", dtype={"Geographic Area(s)": "string"}) # F
bl = bl.rename(columns={ # Rename columns for easier access
    "Geographic Area(s)": "zip",
    "Year(s)": "year",
    "Number of Children Tested": "n_tested",
    "Number of Children with an EBLI": "n_ebli",
    "Percent of Children Tested with an EBLI": "pct_ebli",
    "Blood Lead Reference Value": "bli_threshold"
})
# strip ** from ZIP, keep as 5-digit string
bl["zip"] = bl["zip"].str.replace(r"\*", "", regex=True).str.zfill(5)

# keep one threshold (e.g., 3.5 µg/dL)
bl = bl[bl["bli_threshold"] == 3.5]
bl = bl[bl['n_tested']>10].reset_index(drop=True) # Robustness check

# convert numeric columns and mark suppressed as NaN
bl["n_tested"] = pd.to_numeric(bl["n_tested"], errors="coerce")
bl["n_ebli"] = pd.to_numeric(bl["n_ebli"], errors="coerce")
bl["pct_ebli"] = pd.to_numeric(bl["pct_ebli"], errors="coerce")

print('Brief Information about Blood Lead Level DF\n')
print(bl.info())
display(bl.head())
```

Brief Information about Blood Lead Level DF

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11679 entries, 0 to 11678
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   zip                   11679 non-null  string
1   year                  11679 non-null  int64
2   Age Group/Status      11679 non-null  object
3   Test Type             11679 non-null  object
4   bli_threshold         11679 non-null  float64
5   n_tested              11679 non-null  float64
6   n_ebli                5826 non-null   float64
7   pct_ebli              5826 non-null   float64
dtypes: float64(4), int64(1), object(2), string(1)
memory usage: 730.1+ KB
None
```

	zip	year	Age Group/Status	Test Type	bll_threshold	n_tested	n_ebll	pct_ebll
0	48001	2010	All Children <6	All Test Types	3.5	150.0	15.0	10.0
1	48001	2011	All Children <6	All Test Types	3.5	156.0	7.0	4.5
2	48001	2012	All Children <6	All Test Types	3.5	167.0	12.0	7.2
3	48001	2013	All Children <6	All Test Types	3.5	168.0	11.0	6.5
4	48001	2014	All Children <6	All Test Types	3.5	144.0	NaN	NaN

```
In [3]: #2) LSL Data
proj = pd.read_csv("ProjectMasterExport_4.24.25 Raw Data_Single Family and Rent

# Standardize text columns
for col in ["TYPE3", "LEAD", "AGENCY", "COMMENTS"]:
    if col in proj.columns:
        proj[col] = proj[col].astype(str).str.upper() # Make an unified format

# Clean ADDRESS: remove float artifact ".0" for access later
proj['ADDRESS'] = proj['ADDRESS'].astype(float).astype(int).astype(str)

# 1) Keep lead-related projects
lead_positive = (
    proj["TYPE3"].str.contains("LSL|LSR|LEAD|ABATE|LSL2|LSL3", case=False, na=False)
    | proj["LEAD"].str.contains("LEAD|ABATE", case=False, na=False)
    | proj["AGENCY"].str.contains("LSL|LSR", case=False, na=False)
    | proj["COMMENTS"].str.contains("LEAD", case=False, na=False)
)
proj_lead = proj[lead_positive].copy()

# 2) Drop rows where COMMENTS mention "DROPPED" since these are not actual projects
proj_completed = proj_lead[
    ~proj_lead["COMMENTS"].str.contains("DROPPED", case=False, na=False)
].copy()

print('Brief Information about filtered project DF\n')
print(proj_completed.info())
display(proj_completed.head())
```

Brief Information about filtered project DF

<class 'pandas.core.frame.DataFrame'>

Index: 782 entries, 5 to 4645

Data columns (total 51 columns):

#	Column	Non-Null Count	Dtype
0	AREA	782 non-null	object
1	STREET	782 non-null	object
2	ADDRESS	782 non-null	object
3	DIR	245 non-null	object
4	UNITS	781 non-null	float64
5	PARCEL.	782 non-null	object
6	TYPE	782 non-null	object
7	TYPE2	174 non-null	object
8	TYPE3	782 non-null	object
9	LEAD	782 non-null	object
10	AGENCY	782 non-null	object
11	PROJECT.	769 non-null	object
12	AGE	713 non-null	float64
13	RACE	781 non-null	object
14	ETHNICITY	782 non-null	bool
15	FEMALEHD	72 non-null	object
16	MEDINC	782 non-null	object
17	FAMSIZE	572 non-null	float64
18	HISTORIC	782 non-null	bool
19	AGENT	773 non-null	object
20	SPEC	759 non-null	object
21	START_YEAR	780 non-null	object
22	END_YEAR	779 non-null	object
23	APP_PROCESS_FEE	42 non-null	float64
24	CDBG_SF	779 non-null	float64
25	LSL_REM	713 non-null	float64
26	LSL_INSP	716 non-null	float64
27	CDBG_MULT	781 non-null	float64
28	WTHR	782 non-null	float64
29	CD_LEAD_INSP	776 non-null	float64
30	CD_HIST	782 non-null	int64
31	HOME_SF	782 non-null	float64
32	HOME_MULT	782 non-null	float64
33	HOME_CHDO	782 non-null	float64
34	HOME_MATCH	782 non-null	float64
35	HOME_PI	782 non-null	float64
36	ESG	782 non-null	int64
37	MSHDA	782 non-null	float64
38	OWNER	782 non-null	float64
39	DPA	782 non-null	float64
40	LHOP	781 non-null	float64
41	REHABTOTAL	778 non-null	float64
42	FY_COMPLT	780 non-null	object
43	FY_TOTAL	781 non-null	float64
44	COMMENTS	782 non-null	object
45	BOX_NO	782 non-null	object
46	DISCHARGED	782 non-null	bool
47	PAYOFF	780 non-null	float64
48	GRANTEXP	181 non-null	object
49	BX_LOCATIO	165 non-null	object

50 ACQ_COSTS 671 non-null float64
 dtypes: bool(3), float64(23), int64(2), object(23)
 memory usage: 301.7+ KB
 None

	AREA	STREET	ADDRESS	DIR	UNITS	PARCEL.	TYPE	TYPE2	TYPE3	LEAD	...
5	CW	HAZEL ST	1104	E	1.0	33-01-01-22-134-311	SF	NaN	EMERG	NO LEAD	...
6	CW	RILEY ST	212	NaN	1.0	33-01-01-28-204-301	SF	NaN	LSL	ABATE	...
18	CW	CLIMAX ST	1012	NaN	1.0	33-01-01-22-130-211	RENTAL	NaN	LSL	ABATE	...
19	CW	HOSMER ST	218	N	3.0	33-01-01-15-153-022	RENTAL	NaN	LSL	ABATE	...
20	CW	MAPLEWOOD AVE	1729	NaN	2.0	33-01-01-21-460-076	RENTAL	NaN	LSL	ABATE	...

5 rows x 51 columns

```
In [4]: project_df = pd.read_csv('proj_completed.csv', quoting=1, dtype={'zip': str})

project_df["Year_complt"] = (
    project_df["FY_COMPLT"] # Change the year for parsing. Used year completed
    .str.extract(r"(\d{2})$")[0]
    .apply(lambda x: int("20" + x) if pd.notna(x) else None)
    .astype("Int64")
)

clean_project_df = project_df[project_df['TYPE'].isin(['SF', 'RENTAL'])]

print('Brief Information about filtered project DF with ZIP codes\n')
print(clean_project_df.info())
display(clean_project_df.head())
```

Brief Information about filtered project DF with ZIP codes

<class 'pandas.core.frame.DataFrame'>

Index: 778 entries, 0 to 781

Data columns (total 54 columns):

#	Column	Non-Null Count	Dtype
0	AREA	778 non-null	object
1	STREET	778 non-null	object
2	ADDRESS	778 non-null	int64
3	DIR	242 non-null	object
4	UNITS	777 non-null	float64
5	PARCEL.	778 non-null	object
6	TYPE	778 non-null	object
7	TYPE2	173 non-null	object
8	TYPE3	778 non-null	object
9	LEAD	778 non-null	object
10	AGENCY	778 non-null	bool
11	PROJECT.	766 non-null	object
12	AGE	709 non-null	float64
13	RACE	777 non-null	object
14	ETHNICITY	778 non-null	bool
15	FEMALEHD	72 non-null	object
16	MEDINC	778 non-null	int64
17	FAMSIZE	568 non-null	float64
18	HISTORIC	778 non-null	bool
19	AGENT	770 non-null	object
20	SPEC	755 non-null	object
21	START_YEAR	776 non-null	object
22	END_YEAR	775 non-null	object
23	APP_PROCESS_FEE	42 non-null	float64
24	CDBG_SF	775 non-null	float64
25	LSL_REM	709 non-null	float64
26	LSL_INSP	712 non-null	float64
27	CDBG_MULT	777 non-null	float64
28	WTHR	778 non-null	float64
29	CD_LEAD_INSP	772 non-null	float64
30	CD_HIST	778 non-null	int64
31	HOME_SF	778 non-null	float64
32	HOME_MULT	778 non-null	float64
33	HOME_CHDO	778 non-null	float64
34	HOME_MATCH	778 non-null	float64
35	HOME_PI	778 non-null	float64
36	ESG	778 non-null	int64
37	MSHDA	778 non-null	float64
38	OWNER	778 non-null	float64
39	DPA	778 non-null	float64
40	LHOP	777 non-null	float64
41	REHABTOTAL	774 non-null	float64
42	FY_COMPLT	776 non-null	object
43	FY_TOTAL	777 non-null	float64
44	COMMENTS	778 non-null	object
45	BOX_NO	778 non-null	object
46	DISCHARGED	778 non-null	bool
47	PAYOFF	776 non-null	float64
48	GRANTEXP	181 non-null	object
49	BX_LOCATIO	164 non-null	object

```

50 ACQ_COSTS      667 non-null    float64
51 full_addr      778 non-null    object
52 zip            777 non-null    object
53 Year_complt     776 non-null    Int64
dtypes: Int64(1), bool(4), float64(23), int64(4), object(22)
memory usage: 313.8+ KB
None

```

	AREA	STREET	ADDRESS	DIR	UNITS	PARCEL.	TYPE	TYPE2	TYPE3	LEAD	...
0	CW	HAZEL ST	1104	E	1.0	33-01-01-22-134-311	SF	NaN	EMERG	NO LEAD	...
1	CW	RILEY ST	212	NaN	1.0	33-01-01-28-204-301	SF	NaN	LSL	ABATE	...
2	CW	CLIMAX ST	1012	NaN	1.0	33-01-01-22-130-211	RENTAL	NaN	LSL	ABATE	...
3	CW	HOSMER ST	218	N	3.0	33-01-01-15-153-022	RENTAL	NaN	LSL	ABATE	...
4	CW	MAPLEWOOD AVE	1729	NaN	2.0	33-01-01-21-460-076	RENTAL	NaN	LSL	ABATE	...

5 rows × 54 columns

Note on project_completed.csv

This CSV file was created based on the LSL's data. I ran another code in VSCode using censusgeocode to retrieve all the ZIP data based on the address information that was given to me. The searching process can take up to 5 minutes so I would not re-run it on here. However, I will paste my code in this markdown cell for your reference.

Code:

```

proj_completed["full_addr"] = (
    proj_completed["ADDRESS"].fillna("").astype(str) + ", " +
    proj_completed["STREET"].fillna("").astype(str) + ", Lansing, MI"
# Combine to make full address so censusgeocode can read it and search
)

def get_zip(addr):
    try:
        result = cg.onelineaddress(addr, returntype='locations') #
        Search process
        if result and len(result) > 0:
            return result['addressComponents']['zip'] #Retrieve only
        ZIP for analysis
        return None
    except Exception:
        return None

```

```
proj_completed['zip'] = proj_completed['full_addr'].apply(get_zip)
```

```
In [5]: # 3) Socioeconomic DF
# 2008-2017
socioeconomic_df1 = pd.read_csv("38528-0003-Data.tsv", sep="\t") # Since the f
socioeconomic_df1["ZCTA10"] = socioeconomic_df1["ZCTA10"].astype(int).astype(st
socioeconomic_df1['period'] = 1

socioeconomic_df1 = socioeconomic_df1.rename(columns={
    'ZCTA10': 'zip',
    'PPOV08_12': 'ppov'
})

# 2016-2020
socioeconomic_df2 = pd.read_csv("38528-0005-Data.tsv", sep="\t")
socioeconomic_df2["ZCTA20"] = socioeconomic_df2["ZCTA20"].astype(int).astype(st
socioeconomic_df2['period'] = 2

socioeconomic_df2 = socioeconomic_df2.rename(columns={
    'ZCTA20': 'zip',
    'PPOV16_20': 'ppov'
})

# 2018-2022
socioeconomic_df3 = pd.read_csv("38528-0008-Data.tsv", sep="\t")
socioeconomic_df3["ZCTA20"] = socioeconomic_df3["ZCTA20"].astype(int).astype(st
socioeconomic_df3['period'] = 3
socioeconomic_df3 = socioeconomic_df3.rename(columns={
    'ZCTA20': 'zip',
    'PPOV': 'ppov'
})
```

```
/tmp/ipykernel_463/2227797625.py:13: DtypeWarning: Columns (1,3) have mixed ty
pes. Specify dtype option on import or set low_memory=False.
```

```
socioeconomic_df2 = pd.read_csv("38528-0005-Data.tsv", sep="\t")
```

```
In [6]: nanda_combined = pd.concat([
    socioeconomic_df1[['zip', 'ppov', 'period']],
    socioeconomic_df2[['zip', 'ppov', 'period']],
    socioeconomic_df3[['zip', 'ppov', 'period']]
]).reset_index(drop=True)

print('Brief Information about combined NaNDA PPOV DF\n')
print(nanda_combined.info())
display(nanda_combined.head())
```


Brief Information about combined NaNDA PPOV DF

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99752 entries, 0 to 99751
Data columns (total 3 columns):
#   Column   Non-Null Count  Dtype
---  -
0   zip      99752 non-null  object
1   ppov     99752 non-null  object
2   period   99752 non-null  int64
dtypes: int64(1), object(2)
memory usage: 2.3+ MB
None
```

	zip	ppov	period
0	01001	.0746420025826	1
1	01002	.2741984724998	1
2	01003	.0126582281664	1
3	01005	.046635184437	1
4	01007	.0601202398539	1

Note on the NaNDA datasets

The data could not be concatenated based on year because these are census data collected over a period of time, which means it is not a yearly number. The best I can do this effectively is to assign period based on the year that the data was collected. The first period is from 2008-2017, the second period is from 2016 to 2020, and the third period is 2018-2022. While there is overlapping, I have tried to eliminate this process when merging this with project_completed. Another period functions was created for the blood lead DF so when inner join is used some duplications created from the overlapping is eliminated.

```
In [7]: # Filter to only focus on Lansing ZIP code
lansing_zip_codes_full = [
    '48901', '48906', '48908', '48909', '48910',
    '48911', '48912', '48913', '48915', '48916',
    '48917', '48918', '48919', '48921', '48922',
    '48924', '48929', '48930', '48933', '48937',
    '48950', '48951', '48956'
]

lansing_nanda = nanda_combined[nanda_combined['zip'].isin(lansing_zip_codes_full)]
print('Brief Information about Lansing-focused NaNDA DF\n')
print(lansing_nanda.info())
display(lansing_nanda.head())
```

Brief Information about Lansing-focused NaNDa DF

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 23 entries, 0 to 22
Data columns (total 3 columns):
#   Column   Non-Null Count  Dtype
---  ---
0   zip      23 non-null     object
1   ppov     23 non-null     object
2   period   23 non-null     int64
dtypes: int64(1), object(2)
memory usage: 680.0+ bytes
None
```

	zip	ppov	period
0	48906	.2355376780033	1
1	48910	.2371057122946	1
2	48911	.2421428114176	1
3	48912	.2908626496792	1
4	48915	.2838895320892	1

```
In [8]: def set_period(year): # Setting period for all other DF so it can be merged with
        if year >= 2010 and year <= 2017:
            return 1
        elif year > 2017 and year <=2020:
            return 2
        elif year > 2020 and year <=2022:
            return 3
```

```
In [9]: # Apply function to set period for the blood level data
bl['period'] = bl['year'].apply(set_period)
bl_cleaned = bl.dropna() # remove any null to ensure robustness
periodized_bl = bl_cleaned.groupby(['zip', 'period']).agg(
    period_n_tested=('n_tested', 'sum'),
    period_n_ebll=('n_ebll', 'sum')
).reset_index()

periodized_bl['avg_pct_ebll']=periodized_bl['period_n_ebll']/periodized_bl['period_n_tested']
display(periodized_bl)
```

	zip	period	period_n_tested	period_n_ebll	avg_pct_ebll
0	48001	1.0	963.0	68.0	0.070613
1	48001	3.0	75.0	0.0	0.000000
2	48002	1.0	82.0	0.0	0.000000
3	48002	2.0	27.0	0.0	0.000000
4	48002	3.0	40.0	0.0	0.000000
...	...	...	...	...	...
1895	49968	1.0	39.0	0.0	0.000000
1896	49968	2.0	18.0	0.0	0.000000
1897	49968	3.0	15.0	0.0	0.000000
1898	49969	1.0	56.0	0.0	0.000000
1899	49969	2.0	25.0	0.0	0.000000

1900 rows × 5 columns

```
In [10]: comb_nand_bl = periodized_bl.merge(nanda_combined,how='inner',on=['zip','period'])
comb_nand_bl['ppov'] = comb_nand_bl['ppov'].astype(float)
display(comb_nand_bl)
```

	zip	period	period_n_tested	period_n_ebll	avg_pct_ebll	ppov
0	48001	1.0	963.0	68.0	0.070613	0.109389
1	48001	3.0	75.0	0.0	0.000000	0.134697
2	48002	1.0	82.0	0.0	0.000000	0.054169
3	48002	2.0	27.0	0.0	0.000000	0.042866
4	48002	3.0	40.0	0.0	0.000000	0.067633
...	...	...	...	...	...	...
1890	49968	1.0	39.0	0.0	0.000000	0.173934
1891	49968	2.0	18.0	0.0	0.000000	0.175136
1892	49968	3.0	15.0	0.0	0.000000	0.190571
1893	49969	1.0	56.0	0.0	0.000000	0.108894
1894	49969	2.0	25.0	0.0	0.000000	0.162532

1895 rows × 6 columns

```
In [11]: avg_comb = comb_nand_bl.groupby('zip').agg(
    avg_pct_ebll=('avg_pct_ebll','mean'),
    avg_ppov=('ppov','mean')
).reset_index()

lsl_zip = project_df['zip'].unique().tolist()
avg_comb['has_lsl_project'] = avg_comb['zip'].isin(lsl_zip)
```

```

avg_comb['in_Lansing'] = avg_comb['zip'].isin(lansing_zip_codes_full)

# Label used for visualizations since the blood lead level has data outside of
conditions = [
    avg_comb['zip'].isin(lansing_zip_codes_full) & (avg_comb['has_lsl_project']
    avg_comb['zip'].isin(lansing_zip_codes_full) & (avg_comb['has_lsl_project']
    ~avg_comb['zip'].isin(lansing_zip_codes_full) & (avg_comb['has_lsl_project']
]

labels = [
    'Lansing + Project',
    'Lansing + No Project',
    'Not Lansing + No Project'
]

avg_comb['label'] = np.select(conditions, labels, default='Other')

lansing_no_project = avg_comb[avg_comb['label'] == 'Lansing + No Project']

avg_comb

```

Out[11]:

	zip	avg_pct_ebll	avg_ppov	has_lsl_project	in_Lansing	label
0	48001	0.035306	0.122043	False	False	Not Lansing + No Project
1	48002	0.000000	0.054889	False	False	Not Lansing + No Project
2	48005	0.000000	0.058916	False	False	Not Lansing + No Project
3	48006	0.062016	0.155164	False	False	Not Lansing + No Project
4	48009	0.010204	0.047122	False	False	Not Lansing + No Project
...	...	...	...	...	...	...
820	49955	0.000000	0.134471	False	False	Not Lansing + No Project
821	49958	0.000000	0.190099	False	False	Not Lansing + No Project
822	49963	0.000000	0.172677	False	False	Not Lansing + No Project
823	49968	0.000000	0.179880	False	False	Not Lansing + No Project
824	49969	0.000000	0.135713	False	False	Not Lansing + No Project

825 rows x 6 columns

RQ1.1: Sub-question: Is there any connection between poverty level and percentage of children with EBLL

```

In [12]: plt.figure(figsize=(10,6))

sns.scatterplot(data=avg_comb,
                x='avg_ppov',
                y='avg_pct_ebll',
                palette='viridis',
                alpha=0.5)

#Filtering to only Lansing to overlay another graph with just Lansing projects
only_Lansing = avg_comb[avg_comb['in_Lansing'] == True]

```

```

sns.scatterplot(data=only_Lansing,
                x='avg_ppov',
                y='avg_pct_ebll',
                color='red',
                s=120,          # larger marker
                marker='*',     # star marker to stand out
                zorder=5,      # draw on top
                label='Lansing')

sns.regplot(data=avg_comb,
            x='avg_ppov',
            y='avg_pct_ebll',
            scatter=False,     # hides regplot's own dots
            color='green')

#Annotate the ZIP code with no project
for _, row in lansing_no_project.iterrows():
    plt.annotate(
        text=f"Lansing's ZIP Code with no LSL project: {str(int(row['zip']))}",
        xy=(row['avg_ppov'], row['avg_pct_ebll']),
        xytext=(row['avg_ppov'] + 0.10, row['avg_pct_ebll'] + 0.2),
        fontsize=9,
        color='black',
        arrowprops=dict(arrowstyle='->', color='black', lw=1.2),
        bbox=dict(boxstyle='round,pad=0.4', facecolor='lightyellow', edgecolor='k')
    )

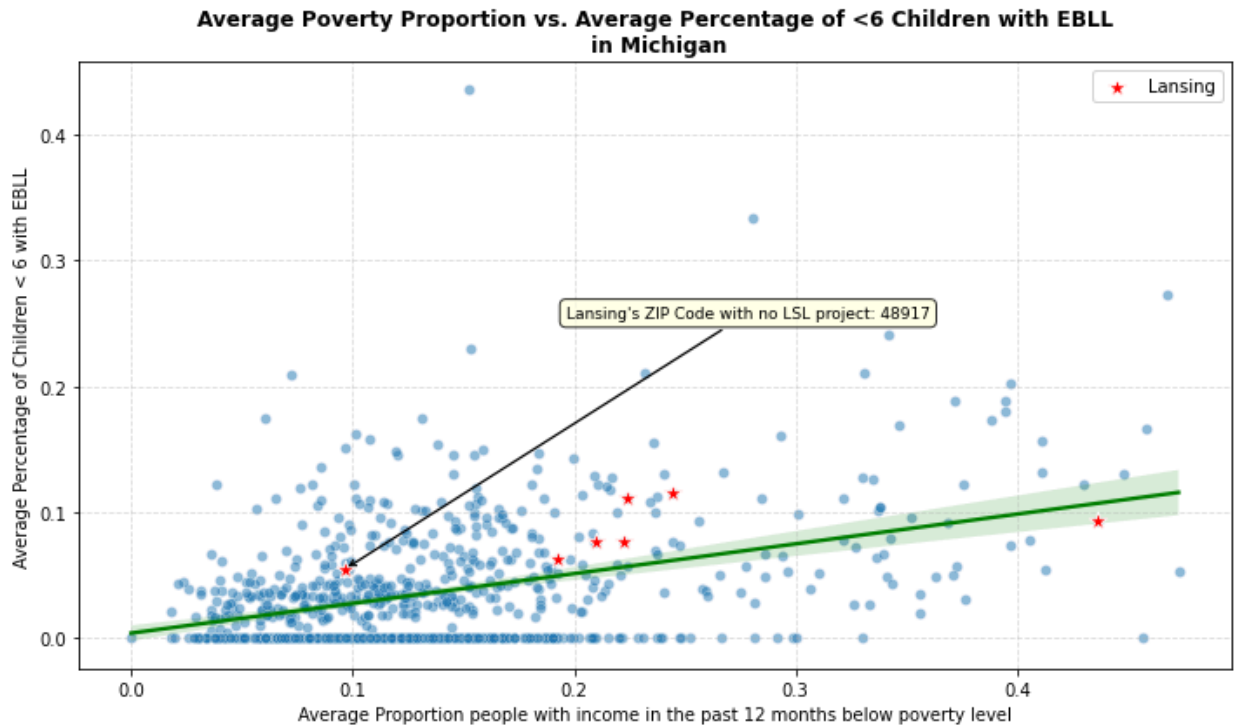
plt.xlabel('Average Proportion people with income in the past 12 months below p
plt.ylabel('Average Percentage of Children < 6 with EBLI')
plt.title('Average Poverty Proportion vs. Average Percentage of <6 Children with
plt.grid(alpha=0.4, ls='--')
plt.tight_layout()
plt.show()

```

```

/usr/local/lib/python3.10/site-packages/seaborn/_oldcore.py:1498: FutureWarning:
is_categorical_dtype is deprecated and will be removed in a future version.
Use isinstance(dtype, CategoricalDtype) instead
    if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/site-packages/seaborn/_oldcore.py:1498: FutureWarning:
is_categorical_dtype is deprecated and will be removed in a future version.
Use isinstance(dtype, CategoricalDtype) instead
    if pd.api.types.is_categorical_dtype(vector):
/tmp/ipykernel_463/2116458011.py:3: UserWarning: Ignoring `palette` because no
`hue` variable has been assigned.
    sns.scatterplot(data=avg_comb,
/usr/local/lib/python3.10/site-packages/seaborn/_oldcore.py:1498: FutureWarning:
is_categorical_dtype is deprecated and will be removed in a future version.
Use isinstance(dtype, CategoricalDtype) instead
    if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/site-packages/seaborn/_oldcore.py:1498: FutureWarning:
is_categorical_dtype is deprecated and will be removed in a future version.
Use isinstance(dtype, CategoricalDtype) instead
    if pd.api.types.is_categorical_dtype(vector):

```



```
In [13]: # Understand the correlation between the two chosen variables
pearson_r, pearson_p = stats.pearsonr(avg_comb['avg_ppov'], avg_comb['avg_pct_ebl'])

print("Pearson r:", pearson_r)
print("p-value:", pearson_p)
```

```
Pearson r: 0.4012237892719163
p-value: 2.9526643197094108e-33
```

```
In [14]: # Run regression analysis to test hypothesis
# H0: There is not relationship between average poverty proportion and average p
# Ha: There is a relationship between average poverty proportion and average pe
# alpha = 0.05

X = sm.add_constant(avg_comb['avg_ppov'])
y = avg_comb['avg_pct_ebl']

# Standard OLS
model = sm.OLS(y, X).fit()

# Robust OLS (HC3 standard errors)
model_robust = sm.WLS(y, X).fit(cov_type='HC3')
print(model_robust.summary())
```

WLS Regression Results

=====						
Dep. Variable:	avg_pct_ebll		R-squared:	0.161		
Model:	WLS		Adj. R-squared:	0.160		
Method:	Least Squares		F-statistic:	91.98		
Date:	Sun, 26 Apr 2026		Prob (F-statistic):	1.01e-20		
Time:	17:15:31		Log-Likelihood:	1413.4		
No. Observations:	825		AIC:	-2823.		
Df Residuals:	823		BIC:	-2813.		
Df Model:	1					
Covariance Type:	HC3					
=====						
	coef	std err	z	P> z	[0.025	0.975]

const	0.0040	0.003	1.282	0.200	-0.002	0.010
avg_ppov	0.2364	0.025	9.591	0.000	0.188	0.285
=====						
Omnibus:	425.947		Durbin-Watson:	1.418		
Prob(Omnibus):	0.000		Jarque-Bera (JB):	4453.530		
Skew:	2.104		Prob(JB):	0.00		
Kurtosis:	13.576		Cond. No.	12.6		
=====						

Notes:

[1] Standard Errors are heteroscedasticity robust (HC3)

```
In [15]: # Re-run WLS with IQR filtered applied

Q1 = avg_comb['avg_pct_ebll'].quantile(0.25)
Q3 = avg_comb['avg_pct_ebll'].quantile(0.75)
IQR = Q3 - Q1

# Remove any datapoint with average_pct_ebll that is beyond range
filtered = avg_comb[
    (avg_comb['avg_pct_ebll'] >= Q1 - 1.5 * IQR) &
    (avg_comb['avg_pct_ebll'] <= Q3 + 1.5 * IQR)
]

X = sm.add_constant(filtered['avg_ppov'])
y = filtered['avg_pct_ebll']
model_no_outliers = sm.WLS(y, X).fit(cov_type='HC3')
print(model_no_outliers.summary())
```

WLS Regression Results

Dep. Variable:	avg_pct_ebll	R-squared:	0.121
Model:	WLS	Adj. R-squared:	0.120
Method:	Least Squares	F-statistic:	91.05
Date:	Sun, 26 Apr 2026	Prob (F-statistic):	1.69e-20
Time:	17:15:31	Log-Likelihood:	1578.9
No. Observations:	793	AIC:	-3154.
Df Residuals:	791	BIC:	-3144.
Df Model:	1		
Covariance Type:	HC3		
=====			
	coef	std err	z
			P> z
			[0.025
			0.975]
const	0.0088	0.002	3.947
avg_ppov	0.1615	0.017	9.542
=====			
Omnibus:	78.462	Durbin-Watson:	1.424
Prob(Omnibus):	0.000	Jarque-Bera (JB):	100.565
Skew:	0.867	Prob(JB):	1.45e-22
Kurtosis:	3.187	Cond. No.	13.4
=====			

Notes:

[1] Standard Errors are heteroscedasticity robust (HC3)

```
In [16]: #Re-run with filtered data
pearson_r, pearson_p = stats.pearsonr(filtered['avg_ppov'], filtered['avg_pct_ebll'])

print("Pearson r:", pearson_r)
print("p-value:", pearson_p)

Pearson r: 0.3475294309655311
p-value: 6.295809338218033e-24
```

Answering the main RQ1

```
In [17]: #Focus on those with high concentration of project (>100 total proj) vs low concentration
proj_agg = (
    project_df
    .groupby(["zip", "Year_complt"])
    .agg(
        n_proj=("TYPE", "count"),
        rehab_total=("REHABTOTAL", "sum")
    )
    .reset_index()
    .rename(columns={"Year_complt": "year"})
).sort_values(by="year")

proj_agg = proj_agg[proj_agg['year'].between(2010, 2022)]

periodized_proj = proj_agg.copy()

periodized_proj['period'] = proj_agg['year'].apply(set_period)

display(periodized_proj)
```


	zip	year	n_proj	rehab_total	period
47	48911	2010	18	122195.789100	1
67	48912	2010	14	213246.000000	1
8	48906	2010	36	906936.394570	1
89	48915	2010	15	214753.578130	1
105	48933	2010	2	34950.000000	1
...	...	...	...	...	...
58	48911	2021	6	81001.799805	3
20	48906	2022	1	740.000000	3
59	48911	2022	1	1325.000000	3
40	48910	2022	2	20244.000000	3
79	48912	2022	1	475.000000	3

75 rows × 5 columns

```
In [18]: # Extract blood level collection for lansing only

bl_lansing_inscope = bl[bl['zip'].isin(lansing_zip_codes_full)][bl['year'].betw
print(bl_lansing_inscope.info())
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 95 entries, 6251 to 6357
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   zip                    95 non-null    string
1   year                   95 non-null    int64
2   Age Group/Status      95 non-null    object
3   Test Type             95 non-null    object
4   bll_threshold          95 non-null    float64
5   n_tested              95 non-null    float64
6   n_ebll                84 non-null    float64
7   pct_ebll              84 non-null    float64
8   period                95 non-null    float64
dtypes: float64(5), int64(1), object(2), string(1)
memory usage: 7.4+ KB
None
```

```
/tmp/ipykernel_463/3892957071.py:3: UserWarning: Boolean Series key will be re
indexed to match DataFrame index.
  bl_lansing_inscope = bl[bl['zip'].isin(lansing_zip_codes_full)][bl['year'].b
etween(2010,2022)]
```

```
In [19]: comb_proj_bl = periodized_proj.merge(bl_lansing_inscope,how='outer',on=['zip',
print(comb_proj_bl.info())
display(comb_proj_bl)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99 entries, 0 to 98
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   zip                    99 non-null     object
1   year                   99 non-null     Int64
2   n_proj                 75 non-null     float64
3   rehab_total            75 non-null     float64
4   period_x               75 non-null     float64
5   Age Group/Status      95 non-null     object
6   Test Type              95 non-null     object
7   bll_threshold          95 non-null     float64
8   n_tested               95 non-null     float64
9   n_ebll                 84 non-null     float64
10  pct_ebll               84 non-null     float64
11  period_y               95 non-null     float64
dtypes: Int64(1), float64(8), object(3)
memory usage: 9.5+ KB
None
```

	zip	year	n_proj	rehab_total	period_x	Age Group/Status	Test Type	bll_threshold	n_tested
0	48911	2010	18.0	122195.78910	1.0	All Children <6	All Test Types	3.5	1270.0
1	48912	2010	14.0	213246.00000	1.0	All Children <6	All Test Types	3.5	372.0
2	48906	2010	36.0	906936.39457	1.0	All Children <6	All Test Types	3.5	808.0
3	48915	2010	15.0	214753.57813	1.0	All Children <6	All Test Types	3.5	338.0
4	48933	2010	2.0	34950.00000	1.0	All Children <6	All Test Types	3.5	43.0
...	...	...	...	...	...	...	...	...	...
94	48933	2012	NaN	NaN	NaN	All Children <6	All Test Types	3.5	39.0
95	48933	2016	NaN	NaN	NaN	All Children <6	All Test Types	3.5	44.0
96	48933	2017	NaN	NaN	NaN	All Children <6	All Test Types	3.5	31.0
97	48933	2021	NaN	NaN	NaN	All Children <6	All Test Types	3.5	13.0
98	48933	2022	NaN	NaN	NaN	All Children <6	All Test Types	3.5	14.0

99 rows × 12 columns

```
In [20]: # Dropna for rows with not pct_ebll due to suppression or missing record

clean_comb_proj_bl = comb_proj_bl.dropna(subset=['n_tested', 'n_ebll', 'pct_ebll'])
clean_comb_proj_bl = clean_comb_proj_bl.fillna(0)
clean_comb_proj_bl.info()
display(clean_comb_proj_bl)
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 84 entries, 0 to 98
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   zip                    84 non-null    object
1   year                   84 non-null    Int64
2   n_proj                 84 non-null    float64
3   rehab_total            84 non-null    float64
4   period_x               84 non-null    float64
5   Age Group/Status       84 non-null    object
6   Test Type              84 non-null    object
7   bll_threshold          84 non-null    float64
8   n_tested               84 non-null    float64
9   n_ebll                 84 non-null    float64
10  pct_ebll               84 non-null    float64
11  period_y               84 non-null    float64
dtypes: Int64(1), float64(8), object(3)
memory usage: 8.6+ KB
```

	zip	year	n_proj	rehab_total	period_x	Age Group/Status	Test Type	bll_threshold	n_tested
0	48911	2010	18.0	122195.78910	1.0	All Children <6	All Test Types	3.5	1270.0
1	48912	2010	14.0	213246.00000	1.0	All Children <6	All Test Types	3.5	372.0
2	48906	2010	36.0	906936.39457	1.0	All Children <6	All Test Types	3.5	808.0
3	48915	2010	15.0	214753.57813	1.0	All Children <6	All Test Types	3.5	338.0
4	48933	2010	2.0	34950.00000	1.0	All Children <6	All Test Types	3.5	43.0
...	...	...	...	...	...	...	...	...	...
92	48917	2021	0.0	0.00000	0.0	All Children <6	All Test Types	3.5	156.0
93	48917	2022	0.0	0.00000	0.0	All Children <6	All Test Types	3.5	200.0
94	48933	2012	0.0	0.00000	0.0	All Children <6	All Test Types	3.5	39.0
95	48933	2016	0.0	0.00000	0.0	All Children <6	All Test Types	3.5	44.0
98	48933	2022	0.0	0.00000	0.0	All Children <6	All Test Types	3.5	14.0

84 rows × 12 columns

```
In [21]: # Aggregate data to ZIP level
avg_project = clean_comb_proj_bl.groupby(['zip'])['n_proj'].mean().reset_index()
avg_project
```

Out [21]:

	zip	n_proj
0	48906	8.916667
1	48909	0.000000
2	48910	12.230769
3	48911	7.461538
4	48912	8.076923
5	48915	6.750000
6	48917	0.000000
7	48933	0.857143

In [22]: *# Label the ZIP to understand the LSL coverage*

```
def label_quintile(df, col='n_proj'):
    df['focus'] = pd.qcut(
        df[col],
        q=5,
        labels=['Q1', 'Q2', 'Q3', 'Q4', 'Q5']
    )
    return df

labeled_zip = label_quintile(avg_project)
```

In [23]: *if 'focus' in clean_comb_proj_bl.columns:*

```
    clean_comb_proj_bl = comb_proj_bl.drop(columns='focus')

merged_concentration = clean_comb_proj_bl.merge(
    labeled_zip[['zip', 'focus']],
    on='zip',
    how='left'
)
```

In [24]: *# Compute rolling avg per focus group to smooth out the curve*

```
merged_concentration = merged_concentration.sort_values('year')
merged_concentration['rolling_avg'] = merged_concentration.groupby('focus')['pct_ebll'].rolling(window=3, min_periods=1).mean()

# Visualization with line plots
fig, ax = plt.subplots(figsize=(10, 6))

# Raw lines (faint)
sns.lineplot(data=merged_concentration, x='year', y='pct_ebll',
             hue='focus', alpha=0.3, linewidth=1, errorbar=None, legend=False, ax=ax)

# Trend lines (bold) – legend=False avoids duplicate legend entries
sns.lineplot(data=merged_concentration, x='year', y='rolling_avg',
             hue='focus', linewidth=2.5, errorbar=None, legend=True, ax=ax)

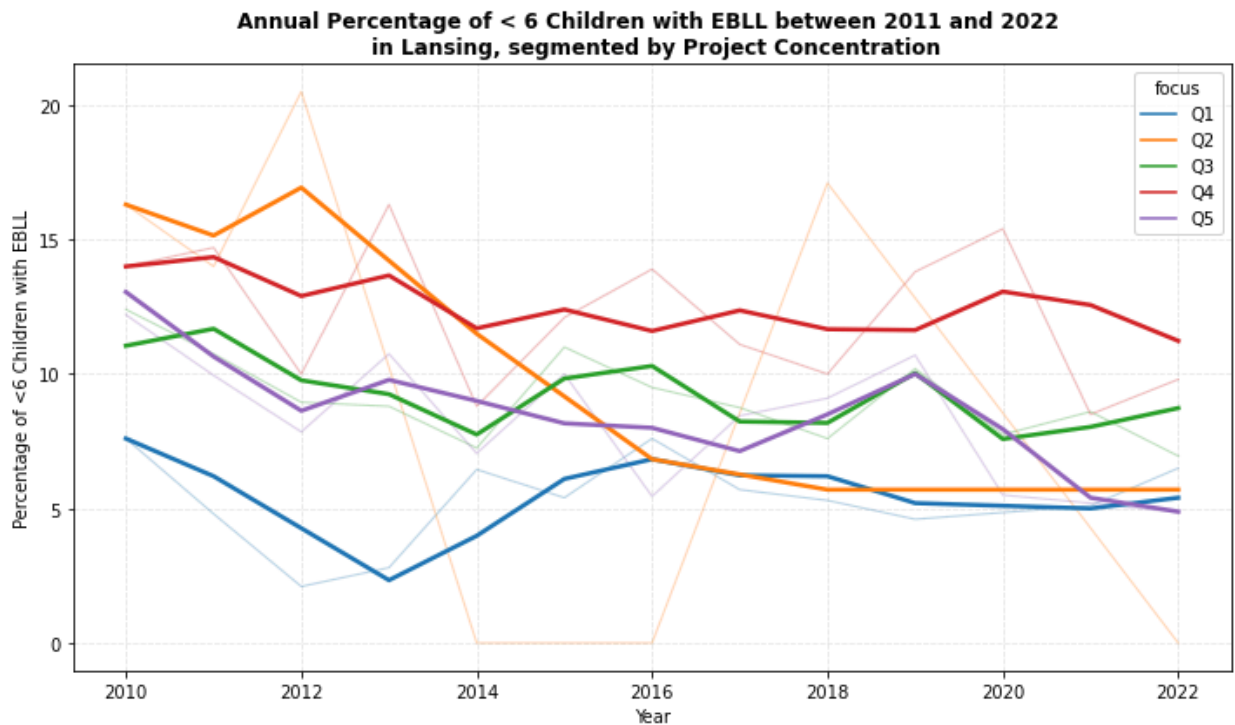
plt.grid(alpha=0.3, ls='--')
plt.xlabel('Year')
plt.ylabel('Percentage of <6 Children with EBL')
plt.title('Annual Percentage of < 6 Children with EBL between 2011 and 2022 \n')
```

```
plt.tight_layout()  
plt.show()
```

```

/tmp/ipykernel_463/799315444.py:3: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.
    merged_concentration['rolling_avg'] = merged_concentration.groupby('focus')
    ['pct_ebl'].transform(
/usr/local/lib/python3.10/site-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, CategoricalDtype) instead
    if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/site-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, CategoricalDtype) instead
    if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/site-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, CategoricalDtype) instead
    if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/site-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.
    with pd.option_context('mode.use_inf_as_na', True):
/usr/local/lib/python3.10/site-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.
    with pd.option_context('mode.use_inf_as_na', True):
/usr/local/lib/python3.10/site-packages/seaborn/_oldcore.py:1057: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.
    grouped_data = data.groupby(
/usr/local/lib/python3.10/site-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, CategoricalDtype) instead
    if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/site-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, CategoricalDtype) instead
    if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/site-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, CategoricalDtype) instead
    if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/site-packages/seaborn/_oldcore.py:1498: FutureWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, CategoricalDtype) instead
    if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/site-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.
    with pd.option_context('mode.use_inf_as_na', True):
/usr/local/lib/python3.10/site-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.
    with pd.option_context('mode.use_inf_as_na', True):
/usr/local/lib/python3.10/site-packages/seaborn/_oldcore.py:1057: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.
    grouped_data = data.groupby(

```

```
In [25]: #Implement ANOVA test to check if there is a difference between the groups
groups = [merged_concentration[merged_concentration['focus'] == q]['pct_ebl']
f_stat, p_val = stats.f_oneway(*groups)
print(f"F-stat: {f_stat:.4f}, p-value: {p_val:.4f}")
```

F-stat: 6.4197, p-value: 0.0002

```
In [26]: #Identify which pair is causing the F-stat to be so much larger
tukey = pairwise_tukeyhsd(
    endog=merged_concentration['pct_ebl'],
    groups=merged_concentration['focus'],
    alpha=0.05
)
print(tukey.summary())
```

Multiple Comparison of Means – Tukey HSD, FWER=0.05

group1	group2	meandiff	p-adj	lower	upper	reject
Q1	Q2	4.5214	0.0691	-0.2213	9.2641	False
Q1	Q3	3.9574	0.0151	0.5374	7.3774	True
Q1	Q4	7.006	0.0	3.0599	10.9522	True
Q1	Q5	3.1774	0.0813	-0.2426	6.5974	False
Q2	Q3	-0.564	0.9964	-4.9451	3.8171	False
Q2	Q4	2.4846	0.6013	-2.3185	7.2877	False
Q2	Q5	-1.344	0.9117	-5.7251	3.0371	False
Q3	Q4	3.0486	0.1182	-0.4547	6.5519	False
Q3	Q5	-0.78	0.9434	-3.6778	2.1178	False
Q4	Q5	-3.8286	0.0251	-7.3319	-0.3253	True

Robustness Checks:

1. Analyzed ZIP codes had at least 10 children tested
2. Interquartile range (IQR) test to filter out outliers.

3. WLS + HC3 were used to compare the non-weighted and weighted versions to account for sample-size effect
4. Limit the use of dropna to avoid Simpson's paradox

RQ2: Among completed Lead Safe Lansing abatement projects completed between 2011 and 2022, are higher rehabilitation costs associated with the inflation rate and the percentage of historic home abated for each year?

Datasets to be used:

1. LSL Projects Dataset
2. FRED's Producer Price Index by Commodity: Inputs to Industries: Net Inputs to Residential Construction, Goods

```
In [27]: # Filter data to year, project_count, and avg_rehab to merge with PPI data
yoy_proj_rehab = merged_concentration.groupby('year').agg(
    project_count=('n_proj', 'sum'),
    avg_rehab=('rehab_total', 'mean')
).reset_index()

yoy_proj_rehab
```

```
Out[27]:
```

	year	project_count	avg_rehab
0	2010	114.0	273596.384499
1	2011	43.0	71757.857143
2	2012	25.0	99908.540038
3	2013	46.0	96961.083327
4	2014	52.0	153590.746588
5	2015	45.0	277470.943700
6	2016	22.0	235865.729077
7	2017	31.0	243937.485035
8	2018	54.0	182508.469867
9	2019	57.0	245051.509767
10	2020	33.0	170056.950000
11	2021	28.0	283796.528076
12	2022	5.0	3254.857143

```
In [28]: # Read in CPI data

ppi = pd.read_csv('WPUIP2311001.csv', parse_dates=['observation_date'])
ppi['observation_date'] = ppi['observation_date'].dt.year
```

```
ppi=ppi.rename(columns={'observation_date':'year','WPUIP2311001':'annual_ppi'})
ppi = ppi.sort_values('year')
ppi['ppi_pct_change'] = ppi['annual_ppi'].pct_change() * 100

print(f'{ppi.info()}\n')
print('PPI Summary')
print(ppi['annual_ppi'].describe())
display(ppi)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 13 entries, 0 to 12
Data columns (total 3 columns):
#   Column          Non-Null Count  Dtype
---  -
0   year            13 non-null    int32
1   annual_ppi      13 non-null    float64
2   ppi_pct_change  12 non-null    float64
dtypes: float64(2), int32(1)
memory usage: 388.0 bytes
None
```

```
PPI Summary
count      13.000000
mean       224.420538
std        32.072459
min        191.250000
25%        207.575000
50%        214.292000
75%        227.867000
max        312.357000
Name: annual_ppi, dtype: float64
```

	year	annual_ppi	ppi_pct_change
0	2010	191.250	NaN
1	2011	202.575	5.921569
2	2012	207.575	2.468222
3	2013	211.100	1.698181
4	2014	214.658	1.685457
5	2015	208.692	-2.779305
6	2016	206.108	-1.238188
7	2017	214.292	3.970734
8	2018	228.308	6.540608
9	2019	227.867	-0.193160
10	2020	226.300	-0.687682
11	2021	266.385	17.713213
12	2022	312.357	17.257728

```
In [29]: #Merge PPI data with yoy_proj_rehab for visualizations effect
ppi_v_rehab = yoy_proj_rehab.merge(ppi,how='inner',on='year')
ppi_v_rehab_clean = ppi_v_rehab[ppi_v_rehab['year']>2010] #Temporarily filterin
```

ppi_v_rehab_clean

Out [29]:

	year	project_count	avg_rehab	annual_ppi	ppi_pct_change
1	2011	43.0	71757.857143	202.575	5.921569
2	2012	25.0	99908.540038	207.575	2.468222
3	2013	46.0	96961.083327	211.100	1.698181
4	2014	52.0	153590.746588	214.658	1.685457
5	2015	45.0	277470.943700	208.692	-2.779305
6	2016	22.0	235865.729077	206.108	-1.238188
7	2017	31.0	243937.485035	214.292	3.970734
8	2018	54.0	182508.469867	228.308	6.540608
9	2019	57.0	245051.509767	227.867	-0.193160
10	2020	33.0	170056.950000	226.300	-0.687682
11	2021	28.0	283796.528076	266.385	17.713213
12	2022	5.0	3254.857143	312.357	17.257728

In [30]:

```

fig, ax1 = plt.subplots(figsize=(10, 6))
ax2 = ax1.twinx()

ax1.plot(ppi_v_rehab['year'], ppi_v_rehab['project_count'],
         color='steelblue', marker='o', linewidth=2, label='Project Count')
ax2.plot(ppi_v_rehab['year'], ppi_v_rehab['ppi_pct_change'],
         color='darkorange', marker='s', linestyle='--', linewidth=2, label='PPI % Change')

#Identify COVID period
ax1.axvspan(2020, 2022, alpha=0.12, color='red', label='COVID-19 Period')

ax1.set_xlabel('Year')
ax1.set_ylabel('Number of Projects', color='steelblue')
ax2.set_ylabel('Annual PPI % Change (%)', color='darkorange')
ax1.tick_params(axis='y', labelcolor='steelblue')
ax2.tick_params(axis='y', labelcolor='darkorange')

lines1, labels1 = ax1.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()

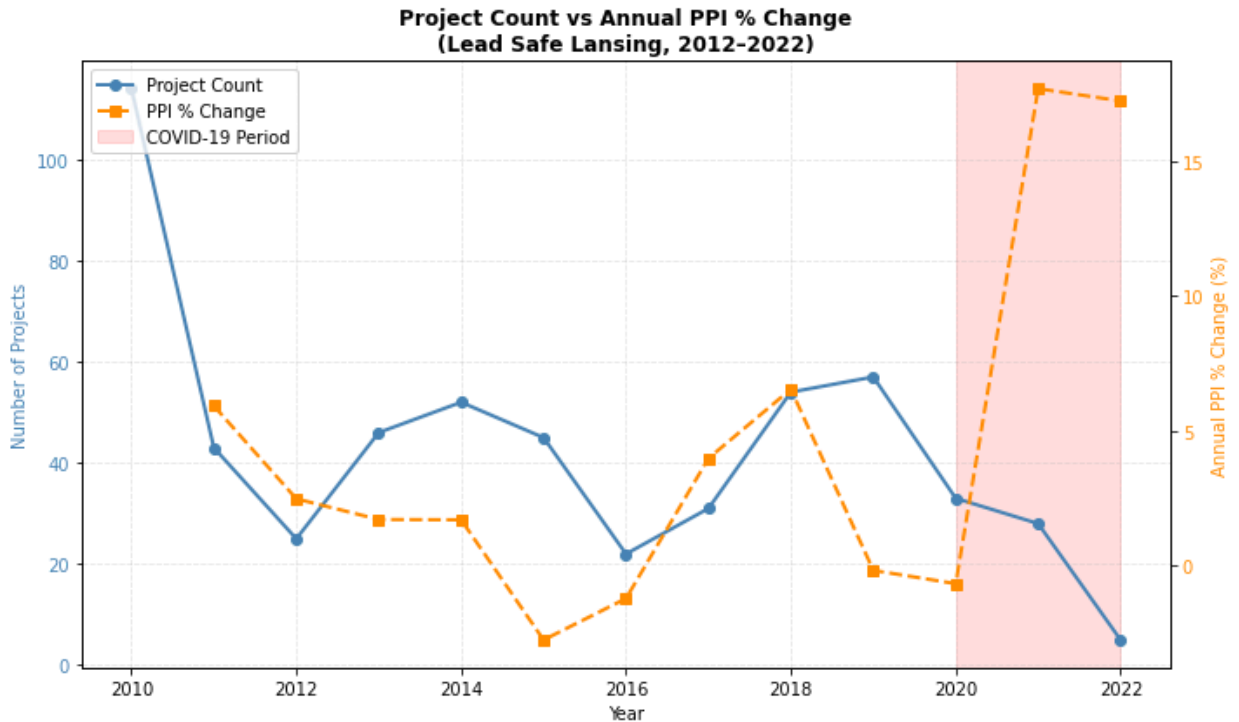
all_lines = lines1 + lines2
all_labels = labels1 + labels2

# Reorder Project Count, PPI % Change, COVID-19 Period so label is in logical order
order = [0, 2, 1]
ax1.legend([all_lines[i] for i in order],
          [all_labels[i] for i in order],
          loc='upper left')

ax1.set_title('Project Count vs Annual PPI % Change\n(Lead Safe Lansing, 2012-2022)',
             fontweight='bold')
ax1.grid(ls='--', alpha=0.3)

```

```
plt.tight_layout()
plt.show()
```



```
In [31]: # Run a Pearson correlation test
ppi_v_rehab_clean_corr = ppi_v_rehab_clean.dropna(subset=['project_count', 'ppi_pct_change'])
pearson_r, pearson_p = stats.pearsonr(ppi_v_rehab_clean_corr['project_count'],
                                       ppi_v_rehab_clean_corr['ppi_pct_change'])

print("Pearson r:", pearson_r)
print("p-value:", pearson_p)
```

```
Pearson r: -0.4983855757835594
p-value: 0.09911711307815718
```

```
In [32]: # Run a Spearman's test since this sample size is small
rho, p = stats.spearmanr(ppi_v_rehab_clean['ppi_pct_change'], ppi_v_rehab_clean['project_count'])
print(f"Spearman rho={rho:.3f}, p={p:.3f}")
```

```
Spearman rho=-0.224, p=0.484
```

```
In [33]: #Carry out another test to check if there is any discrepancy between tests
X = sm.add_constant(ppi_v_rehab_clean['ppi_pct_change'])
y = ppi_v_rehab_clean['avg_rehab']

model = sm.WLS(y,X).fit()

model_robust = sm.WLS(y, X).fit(cov_type='HC3')
print(model_robust.summary())
```

WLS Regression Results

```

=====
Dep. Variable:          avg_rehab    R-squared:                0.082
Model:                  WLS          Adj. R-squared:           -0.010
Method:                 Least Squares    F-statistic:              0.1568
Date:                  Sun, 26 Apr 2026    Prob (F-statistic):       0.700
Time:                  17:15:36          Log-Likelihood:           -152.81
No. Observations:      12              AIC:                     309.6
Df Residuals:          10              BIC:                     310.6
Df Model:               1
Covariance Type:       HC3
=====

```

```

=====
              coef      std err          z      P>|z|      [0.025      0.
975]
-----
const          1.886e+05    2.7e+04     6.977     0.000    1.36e+05    2.42
e+05
ppi_pct_change -3798.9623    9592.697    -0.396     0.692   -2.26e+04    1.5
e+04
=====

```

```

=====
Omnibus:                0.436    Durbin-Watson:              1.736
Prob(Omnibus):           0.804    Jarque-Bera (JB):          0.520
Skew:                    0.252    Prob(JB):                  0.771
Kurtosis:                2.114    Cond. No.                  9.45
=====

```

Notes:

[1] Standard Errors are heteroscedasticity robust (HC3)

```

/usr/local/lib/python3.10/site-packages/scipy/stats/_stats_py.py:1806: UserWarning: kurtosistest only valid for n>=20 ... continuing anyway, n=12
warnings.warn("kurtosistest only valid for n>=20 ... continuing ")

```

```

In [34]: #Checking for outliers and check if it should be removed
Q1 = ppi_v_rehab['ppi_pct_change'].quantile(0.25)
Q3 = ppi_v_rehab['ppi_pct_change'].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = ppi_v_rehab[(ppi_v_rehab['ppi_pct_change'] < lower) |
                        (ppi_v_rehab['ppi_pct_change'] > upper)]
print(outliers[['year', 'ppi_pct_change', 'avg_rehab']])

   year  ppi_pct_change  avg_rehab
11  2021      17.713213 283796.528076
12  2022      17.257728  3254.857143

```

Note about the outlier test

While I understand that both years are outliers that should be removed out of the DF, I decided to keep 2022 since it was following well with the trend. 2021 was the only that seems anomalous to me.

```
In [35]: # Removing 2021
ppi_v_rehab_wo2021 = ppi_v_rehab[ppi_v_rehab['year']!=2021]
```

```
In [36]: #Create a graph to demonstrate the association between PPI and mean rehabilitat
fig, ax = plt.subplots(figsize=(10, 6))

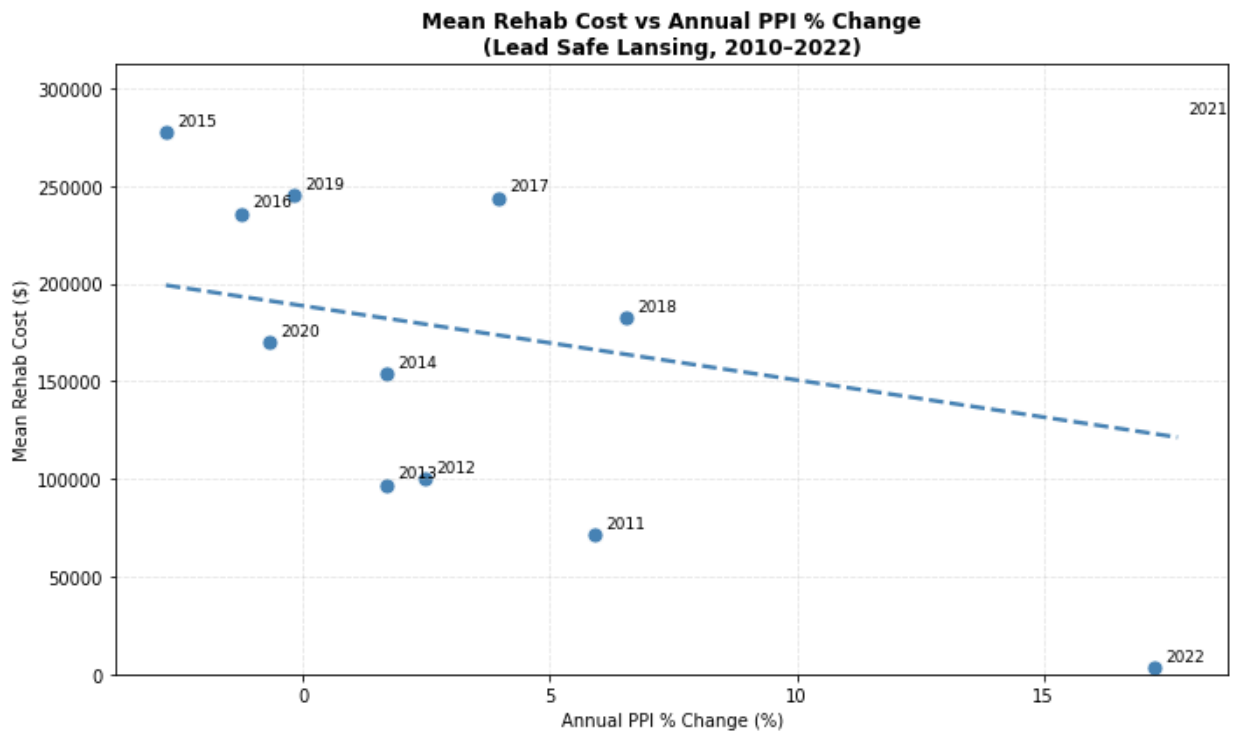
sns.scatterplot(data=ppi_v_rehab_wo2021, x='ppi_pct_change', y='avg_rehab',
                color='steelblue', s=80, ax=ax)
sns.regplot(data=ppi_v_rehab_clean, x='ppi_pct_change', y='avg_rehab',
            scatter=False, ci=False,
            line_kws={'color': 'steelblue', 'linestyle': '--'},
            ax=ax)

# Year labels
for _, row in ppi_v_rehab_clean.iterrows():
    ax.annotate(str(int(row['year'])),
                (row['ppi_pct_change'], row['avg_rehab']),
                textcoords='offset points', xytext=(6, 4), fontsize=9)

ax.set_title('Mean Rehab Cost vs Annual PPI % Change\n(Lead Safe Lansing, 2010-
              fontweight='bold')
ax.set_xlabel('Annual PPI % Change (%)')
ax.set_ylabel('Mean Rehab Cost ($)')
ax.grid(ls='--', alpha=0.3)
ax.set_ylim(0, ppi_v_rehab['avg_rehab'].max() * 1.1)

plt.tight_layout()
plt.show()
```

```
/usr/local/lib/python3.10/site-packages/seaborn/_oldcore.py:1498: FutureWarning:
is_categorical_dtype is deprecated and will be removed in a future version.
Use isinstance(dtype, CategoricalDtype) instead
  if pd.api.types.is_categorical_dtype(vector):
/usr/local/lib/python3.10/site-packages/seaborn/_oldcore.py:1498: FutureWarning:
is_categorical_dtype is deprecated and will be removed in a future version.
Use isinstance(dtype, CategoricalDtype) instead
  if pd.api.types.is_categorical_dtype(vector):
```



```
In [37]: #Re-run test with no null value and outliers removed
clean_ppi_rehab_wo2021 = ppi_v_rehab_wo2021.dropna(subset=['ppi_pct_change'])
pearson_r, pearson_p = stats.pearsonr(clean_ppi_rehab_wo2021['avg_rehab'], clean_ppi_rehab_wo2021['ppi_pct_change'])

print("Pearson r:", pearson_r)
print("p-value:", pearson_p)

Pearson r: -0.7405676258662323
p-value: 0.009139180873648968
```

```
In [38]: #Re-run test with no null value and outliers removed
rho, p = stats.spearmanr(clean_ppi_rehab_wo2021['ppi_pct_change'], clean_ppi_rehab_wo2021['avg_rehab'])
print(f"Spearman rho={rho:.3f}, p={p:.3f}")

Spearman rho=-0.627, p=0.039
```

```
In [39]: #Re-run test with no null value and outliers removed
X = sm.add_constant(clean_ppi_rehab_wo2021['ppi_pct_change'])
y = clean_ppi_rehab_wo2021['avg_rehab']

model = sm.WLS(y, X).fit()

model_robust = sm.WLS(y, X).fit(cov_type='HC3')
print(model_robust.summary())
```


WLS Regression Results

Dep. Variable:	avg_rehab	R-squared:	0.548
Model:	WLS	Adj. R-squared:	0.498
Method:	Least Squares	F-statistic:	32.50
Date:	Sun, 26 Apr 2026	Prob (F-statistic):	0.000294
Time:	17:15:38	Log-Likelihood:	-135.73
No. Observations:	11	AIC:	275.5
Df Residuals:	9	BIC:	276.2
Df Model:	1		
Covariance Type:	HC3		

	coef	std err	z	P> z	[0.025	0.975]
const	1.983e+05	2.06e+04	9.611	0.000	1.58e+05	2.39e+05
ppi_pct_change	-1.159e+04	2032.422	-5.701	0.000	-1.56e+04	-7602.564

Omnibus:	1.594	Durbin-Watson:	0.696
Prob(Omnibus):	0.451	Jarque-Bera (JB):	0.778
Skew:	0.023	Prob(JB):	0.678
Kurtosis:	1.698	Cond. No.	7.19

Notes:

[1] Standard Errors are heteroscedasticity robust (HC3)

/usr/local/lib/python3.10/site-packages/scipy/stats/_stats_py.py:1806: UserWarning: kurtosistest only valid for n>=20 ... continuing anyway, n=11
 warnings.warn("kurtosistest only valid for n>=20 ... continuing ")

```
In [40]: # Use Bootstrap resample to check there is any discrepancy if data is resampled
np.random.seed(42) #arbitrary number

n_inter = 10000

observed_r, observed_p = stats.pearsonr(
    clean_ppi_rehab_wo2021['ppi_pct_change'],
    clean_ppi_rehab_wo2021['avg_rehab']
)

bootstrap_rs = []

for _ in range(n_inter):
    sample = clean_ppi_rehab_wo2021.sample(n=len(ppi_v_rehab_clean), replace=True)
    r, _ = stats.pearsonr(sample['ppi_pct_change'], sample['avg_rehab'])
    bootstrap_rs.append(r)

ci_low, ci_high = np.percentile(bootstrap_rs, [2.5, 97.5])

print(f'Observation Pearson r: {observed_r:.3f}')
print(f'Parametric p-value: {observed_p:.3f}')
print(f'Bootstrap 95% CI: [{ci_low:.3f}, {ci_high:.3f}]')
print(f'CI contains zero: {ci_low < 0 < ci_high}')
```

Observation Pearson r: -0.741
 Parametric p-value: 0.009
 Bootstrap 95% CI: [-0.917, -0.174]
 CI contains zero: False

```
In [41]: # Since there is a negative relationship between PPI and rehabilitation cost, a

# Relationship between avg_rehab and project_count
# Weighted by project_count to account for years with fewer projects
wls_model = smf.wls('avg_rehab ~ project_count',
                    data=ppi_v_rehab_clean,
                    weights=ppi_v_rehab_clean['project_count']).fit()

wls_hc3 = wls_model.get_robustcov_results(cov_type='HC3')
print(wls_hc3.summary())
```

WLS Regression Results

```
=====
Dep. Variable:          avg_rehab      R-squared:                0.002
Model:                  WLS           Adj. R-squared:           -0.097
Method:                 Least Squares  F-statistic:              0.02588
Date:                   Sun, 26 Apr 2026 Prob (F-statistic):       0.875
Time:                   17:15:48       Log-Likelihood:          -152.21
No. Observations:      12             AIC:                     308.4
Df Residuals:          10             BIC:                     309.4
Df Model:               1
Covariance Type:       HC3
=====
```

```
===
               coef      std err          t      P>|t|      [0.025      0.9
75]
```

```
-----
Intercept      1.701e+05   8.23e+04     2.067     0.066   -1.33e+04   3.53e
+05
project_count   299.9819   1864.608     0.161     0.875   -3854.624   4454.
587
```

```
=====
Omnibus:                2.040    Durbin-Watson:              1.292
Prob(Omnibus):           0.361    Jarque-Bera (JB):          0.890
Skew:                   -0.061    Prob(JB):                  0.641
Kurtosis:                1.671    Cond. No.                  166.
=====
```

Notes:

[1] Standard Errors are heteroscedasticity robust (HC3)

```
/usr/local/lib/python3.10/site-packages/patsy/util.py:672: FutureWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, CategoricalDtype) instead
    return _pandas_is_categorical_dtype(dt)
/usr/local/lib/python3.10/site-packages/patsy/util.py:672: FutureWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, CategoricalDtype) instead
    return _pandas_is_categorical_dtype(dt)
/usr/local/lib/python3.10/site-packages/patsy/util.py:672: FutureWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, CategoricalDtype) instead
    return _pandas_is_categorical_dtype(dt)
/usr/local/lib/python3.10/site-packages/scipy/stats/_stats_py.py:1806: UserWarning: kurtosistest only valid for n>=20 ... continuing anyway, n=12
    warnings.warn("kurtosistest only valid for n>=20 ... continuing ")
```

Robustness Checks:

1. Interquartile range (IQR) test to filter out outliers
2. Bootstrap Resample (10,000 iterations) to avoid the impact of small sample
3. Test multiple connections to reveal any check any confounding variables

In []: